CHAPTER 1: INTRODUCTION

Introduction

In the modern era of digital learning, technology plays a crucial role in transforming how students access, share, and consume educational content. With increasing academic pressure and a vast syllabus, students need organized, reliable, and syllabus-specific resources that are easily accessible anytime, anywhere. The SmartStudy web application is designed to address these needs by creating a structured, interactive, and collaborative online study platform tailored specifically for Mumbai University students pursuing B.Sc. in Information Technology.

1.1 Background

In today's digital era, access to structured, syllabus-aligned learning resources is crucial for academic success — especially for students pursuing Information Technology (IT) and Computer Science under Mumbai University. Students often rely on scattered PDFs, handwritten notes, or unreliable online content that fails to match the university syllabus.

SmartStudy addresses this problem by offering a centralized web-based platform that organizes syllabus-specific notes, interactive quizzes, and other study materials. It simplifies self-study and exam preparation while promoting collaboration between students and professionals.

The platform blends accessibility, interactivity, and structure, allowing users to study efficiently and engage with authentic, syllabus-oriented materials.

1.2 Objectives

The main objectives of the SmartStudy web application are:

• Develop an online platform that streamlines how students and authors share and access educational material.
• Build a responsive, user-friendly interface for all roles.
• Provide syllabus-specific notes, topic-wise quizzes, and search capabilities.
• Enable authors or professionals to upload verified content via professional profiles.

• Allow students to explore freely and log in for personalized use.
• Maintain data security through authentication and role-based permissions.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of SmartStudy is to bridge the gap between structured learning and open collaboration by offering a unified digital platform where students can access reliable, syllabus-based resources and professionals can share their expertise. It replaces scattered and inconsistent materials with a centralized hub featuring syllabus-specific notes, topic-wise quizzes, and verified uploads — ensuring both accessibility and academic credibility.

Beyond being just a study tool, SmartStudy promotes an interactive learning culture where students develop self-discipline, curiosity, and conceptual clarity. By integrating technology with education, the platform supports independent learning while maintaining quality standards set by universities. Its ultimate goal is to make knowledge-sharing efficient, trustworthy, and beneficial for both learners and contributors.

1.3.2 Scope

SmartStudy is a full-stack web application that supports multiple user roles:

• Student Login: Access to syllabus-aligned notes, quizzes, and self-study materials.
• Author / Professional Login: Upload, manage, and organize verified study content through professional profiles.
• Guest Users: Browse and preview topics or sample quizzes; login required for full access.

The platform currently caters to the Mumbai University IT/CS syllabus but can easily expand to include other subjects, universities, and even national-level curricula.Core modules include authentication, content upload/download, quiz management, search, and feedback. Future improvements may include AI-driven recommendations, discussion forums, and a mobile application for on-the-go learning.

This adaptability ensures SmartStudy can promote a sustainable digital learning ecosystem across various disciplines and institutions. The architecture is designed to support the eventual integration

of advanced analytics to help authors understand content usage and student performance trends. The system also maintains a flexible structure to accommodate features like gamification elements and certification tracking for skill development.

This commitment to continuous improvement guarantees that SmartStudy remains a cutting-edge resource for learners and educators alike.

1.3.3 Applicability

The SmartStudy web application is primarily applicable to undergraduate students of Mumbai University's IT and Computer Science programs. It provides a centralized, accessible, and syllabus-aligned platform that supports self-learning and complements traditional classroom teaching.

Additionally, the system benefits authors and professionals by offering a structured space to publish and manage educational materials. It encourages collaboration, feedback exchange, and continuous academic improvement.

With its scalable and adaptable design, SmartStudy can be expanded to other disciplines and institutions, promoting a sustainable digital learning ecosystem.

1.4 Achievements

Developing SmartStudy offered extensive hands-on experience in full-stack web development and provided valuable insight into real-world software engineering practices. The project served as a practical application of theoretical knowledge gained throughout the course, bridging the gap between classroom learning and professional development. Key accomplishments include:

• Designing a responsive and user-friendly interface using modern technologies such as Django, React, and SQLite.
• Implementing secure authentication and role-based access control to manage different user roles like students, authors, and guests.
• Applying structured software engineering methodologies — from requirement analysis and design to implementation and testing.
• Strengthening knowledge of database modeling, data validation, and system optimization for efficiency and scalability.

• Enhancing understanding of frontend–backend integration, API handling, and UI/UX consistency.

• Developing team coordination, time management, and project documentation skills essential for professional software projects.The project not only demonstrates technical proficiency but also highlights how academic concepts can be transformed into a functional, user-centered solution that supports both learners and educators in the digital education space.

1.5 Organisation of Report

This report is organized into **eight chapters**, each focusing on a key phase of the project development:

Chapter 1: Introduction — Provides an overview of the project, including background, objectives, purpose, scope, applicability, and achievements.

Chapter 2: Survey of Technologies — Reviews existing technologies, frameworks, and tools relevant to web-based educational platforms and collaborative learning such as Django, React, and SQLite.

Chapter 3: Requirements and Analysis — Discusses the problem definition, requirements gathering, system requirements, planning and scheduling, and detailed system analysis.

Chapter 4: System Design — Describes the system architecture, module structure, database design, data flow diagrams, and overall system modeling.

Chapter 5: Implementation and Testing — Explains the development process, coding environment, testing strategies, and verification of system functionality.

Chapter 6: Results and Discussion — Presents the results obtained after implementing the system, including output screens, system performance, and discussion of outcomes.

Chapter 7: Conclusion and Future Work — Summarizes the overall findings of the project, discusses system limitations, and proposes possible future enhancements.

Chapter 8: References — Lists all books, research papers, websites, and other sources referred to during the development and documentation of the project.

CHAPTER 2: SURVEY OF TECHNOLOGIES

2.1 Introduction

Developing SmartStudy required careful selection of technologies that are robust, widely used, and beginner-friendly. The aim was to build a platform that is efficient, interactive, and easy to maintain.

For this project, the chosen technologies are:

Django (Python Web Framework) — backend development

Django REST Framework (DRF) — API development

React — frontend development

SQLite — database management

This chapter demonstrates an understanding of these technologies, explains their working, and highlights real-world usage. A comparative study of alternative technologies is included to justify the choices made for SmartStudy.

2.2 Backend Technology: Django

Django is a high-level Python web framework designed to enable rapid development of secure and maintainable web applications. It follows the Model-View-Template (MVT) architecture, which separates data management, business logic, and presentation for better organization and scalability.

Features and Examples

User Authentication:
Built-in registration, login, and permission systems support multiple roles like students and authors.

Admin Dashboard:
Provides a ready-made interface for administrators and authors to upload and manage study materials.

ORM (Object-Relational Mapper):
Allows Python-based interaction with the database instead of writing complex SQL queries.

Real-world Use:
Django powers large-scale websites such as Instagram, Pinterest, and Mozilla, demonstrating reliability and scalability.

Why Django?

Speeds up development with prebuilt tools

Strong security features to protect user data

Efficient handling of multiple user roles and permissions

Simplifies database operations through ORM integration

2.3 API Technology: Django REST Framework (DRF)

Modern web applications separate backend and frontend using APIs. Django REST Framework simplifies creating RESTful APIs within Django applications.

Features and Examples

Authentication & Permissions:
Controls resource access — for example, only authors can upload notes, while students can view and rate them.

Serialization:
Converts database objects into JSON format so that the frontend application can easily consume the data.

Browsable API:
Provides a web interface for testing APIs during development.

Real-world Use

Many educational platforms and web applications use Django with DRF to build scalable and interactive services.

Why DRF?

Seamless integration with Django backend models

Easy implementation of secure API endpoints

Extensive documentation and strong community support

2.4 Frontend Technology: ReactJS

React is a JavaScript library used for building interactive and responsive user interfaces. It follows a component-based architecture that allows developers to build modular and reusable UI components.

Features and Examples

Component-Based Architecture:
The interface is divided into reusable components such as search bars, quiz sections, dashboards, and note displays.

Virtual DOM:
Optimizes updates to the user interface, resulting in faster performance.

Rich Ecosystem:
Supports tools for routing, state management, and UI enhancements.

Real-world Use

React is widely used in platforms like Facebook, Netflix, and WhatsApp Web due to its ability to create dynamic and responsive user interfaces.

Why ReactJS?

Provides smooth and responsive user experience

Encourages modular and maintainable code

Large developer community and learning resources

2.5 Database Technology: SQLite

SQLite is a lightweight, serverless relational database management system. It stores structured data in a single file and is commonly used for web applications during development or small-to-medium scale deployments.

Features and Examples

Structured Data Storage:
Stores information such as student profiles, study notes, quizzes, and ratings.

Lightweight Architecture:
Does not require a separate server process and integrates directly with the application.

Compatibility:
Works seamlessly with Django's ORM, enabling developers to interact with the database using Python models.

Why SQLite?

Easy setup with no separate server configuration

Fully supported by Django's default configuration

Efficient for academic projects and small-scale applications

Simplifies database management during development

2.6 Technology Comparison & Justification

Component	Technology Chosen	Alternatives Considered	Justification
Frontend	ReactJS	Angular, VueJS	Reusable components, fast rendering, large community
Backend	Django	Flask, NodeJS	Built-in authentication, ORM, rapid development
Database	SQLite	MySQL, PostgreSQL	Lightweight, simple configuration, Django default
API	REST	GraphQL	Simple CRUD operations and easy integration
UI Styling	HTML5, CSS3, Bootstrap / Tailwind	Pure CSS	Faster responsive design and consistent layout

Justification Summary

The selected technologies provide a balanced combination of simplicity, scalability, and security. Each technology was chosen based on its compatibility with the project requirements and its ability to deliver a reliable and maintainable solution for the SmartStudy platform.

2.7 Conclusion

SmartStudy uses Django, Django REST Framework, React, and SQLite to create a secure, interactive, and efficient e-learning platform. This technology stack simplifies content management and offers a responsive user interface, making it a practical tool for Mumbai University IT and Computer Science students for collaborative learning and exam preparation.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Students at Mumbai University, particularly in IT and Computer Science, face significant challenges in accessing well-organized, syllabus-specific study materials and interactive self-study tools. Traditional methods such as scattered class notes, reference books, or unstructured online resources often lack alignment, flexibility, and engagement required for effective learning.

The absence of a centralized, reliable, and interactive study portal prevents students from preparing efficiently for exams and limits peer collaboration. Similarly, teachers and authors lack a streamlined digital platform to upload, manage, and distribute high-quality, syllabus-aligned content.

The main problem addressed by SmartStudy is the lack of a personalized, syllabus-oriented, and collaborative digital platform that integrates structured content, quizzes, peer-to-peer knowledge sharing, and additional learning tools.

Sub-Problems

Access to Structured Content:
Students struggle to find topic-wise, syllabus-aligned materials. Online sources are fragmented or not tailored to the university curriculum.

Exam Readiness Assessment:
Traditional methods do not provide interactive practice tools such as MCQs, quizzes, or mock tests to track performance.

Collaborative Learning Gap:
Peer-to-peer collaboration, including notes sharing, discussions, and content rating, is poorly supported.

Content Management for Educators:
Teachers or authors face difficulties in securely uploading and managing their academic materials.

3.2 Requirements Specification

The SmartStudy system is designed to fulfill the following requirements.

Functional Requirements

User Management:
Registration, login, profile management, and role-based access (Student, Author, Admin).

Study Material Module:
Uploading, downloading, rating, and organized display of syllabus-aligned notes, tutorials, and examples.

Exam Preparation Module:
Quizzes, MCQs, timed tests, result storage, and performance tracking.

Knowledge Search Module:
Topic-based search for efficient and personalized learning.

Feedback System:
Students can rate and comment on content to improve quality.

Admin Management:
Admins can monitor user activities, manage uploaded content, and ensure proper functioning of the system.

Non-Functional Requirements

Usability:
Intuitive, responsive interface for desktop and mobile browsers.

Performance:
Quick loading times and efficient search across study materials.

Scalability:
Ability to add new courses, semesters, or modules without redesign.

Security:

Secure login system, role-based access control, and encrypted user data.

Reliability:

Consistent system uptime and accurate retrieval of content.

3.3 Planning and Scheduling

SmartStudy will be developed in the following phases:

Requirement Gathering & Analysis – Identify the needs of students and educators.

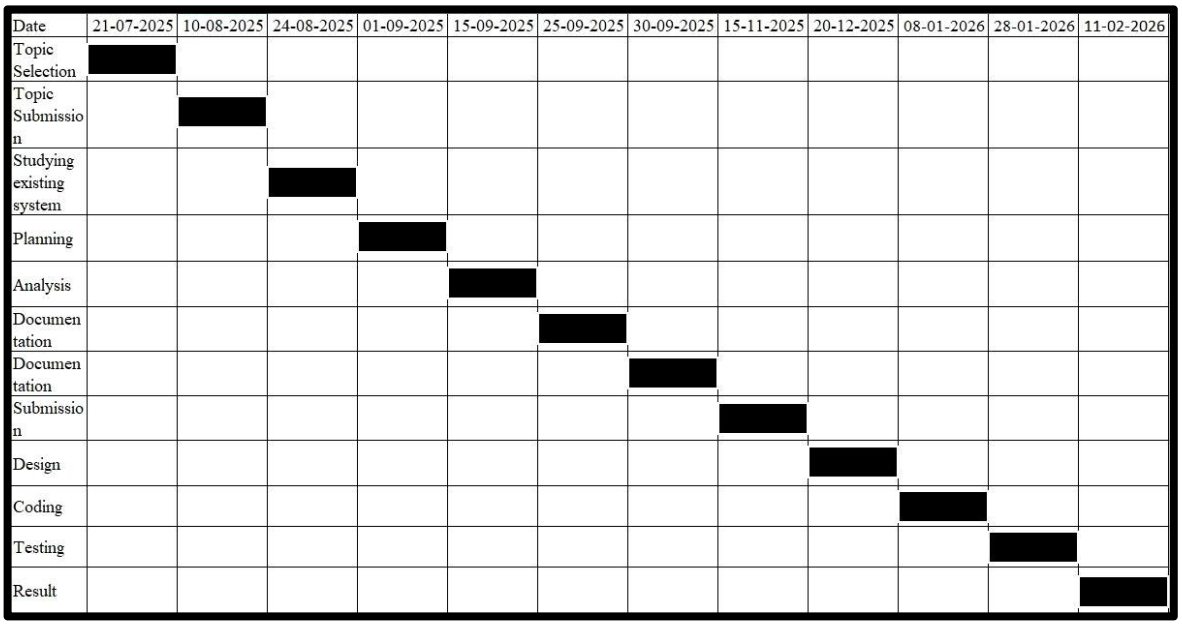
System Design – Prepare architecture diagrams, database schema, and UI mockups.

Implementation – Develop backend using Django and Django REST Framework, frontend using React, and database using SQLite.

Testing – Conduct unit testing, integration testing, and user acceptance testing.

Deployment & Documentation – Host the web application and prepare manuals.

3.4 Gantt Chart



At its heart, a Gantt chart is a horizontal bar chart that represents a project schedule. It maps out **what** needs to be done and **when**.

The Horizontal Axis: Represents time (days, weeks, or months).

The Vertical Axis: Lists the specific tasks or activities.

The Bars: The length of each bar represents the duration of a task. The position indicates the start and end dates.

Key Features of a Gantt Chart :

To understand a Gantt chart at an advanced level, you need to look beyond just the bars:

Task Dependencies: Not all tasks can start at once. Dependencies (shown as arrows connecting bars) indicate that Task B cannot start until Task A is finished.

Milestones: Represented by diamond shapes, these are significant checkpoints or "win" states in a project (e.g., "Design Approved") that have zero duration.

The Critical Path: This is the sequence of tasks that determines the shortest possible duration of the project. If a task on the critical path is delayed, the whole project is delayed.

3.5 Software and Hardware Requirements

Hardware Requirements

Processor: Intel i5 or higher

RAM: 8 GB minimum

Storage: 256 GB SSD

Peripherals: Keyboard, Mouse, Monitor

Internet: Stable broadband connection

Software Requirements

Operating System: Windows 10/11 or Linux Ubuntu

Backend Framework: Django with Django REST Framework

Frontend: React

Database: SQLite

Development Tools: Visual Studio Code, Postman (API testing), GitHub (version control)

Browser: Chrome / Edge / Firefox for testing

3.6 Preliminary Product Description

The SmartStudy platform will provide:

• A centralized web application with syllabus-specific study materials.
• Role-based dashboards for students, authors, and administrators.
• Quizzes, MCQs, and performance reports for self-assessment.
• Peer-driven knowledge sharing with notes rating and feedback system.
• Admin monitoring for managing content and user activities.

Potential future enhancements include AI-based study recommendations, discussion forums, and mobile application support.

3.7 Conceptual Models

The conceptual model represents the logical structure of the SmartStudy System, outlining the main entities, their attributes, and interactions. It illustrates how different user roles — Student, Author, and Admin — interact with the platform's academic features and system data.

User

The User entity represents every individual interacting with SmartStudy. Each user is uniquely identified by login credentials and authenticated through a secure authentication mechanism.

The system classifies users into three roles:

Student
Students can access study materials, attempt quizzes, and provide feedback or ratings.

Author
Authors upload notes, create quizzes, and maintain academic content quality.

Admin
Admins supervise user activities, verify uploaded content, manage accounts, and ensure the system runs securely.

Study Material (Notes)

This entity contains syllabus-aligned learning resources uploaded by Authors.

Attributes include:

Title
Subject
Description
File Path
UploadedBy
Rating

Students can view, download, and rate materials.

Quiz

The Quiz entity represents interactive assessments created by Authors.

Key attributes include:

QuizID
Title
Subject
Questions
CreatedBy
DateCreated

Students attempt quizzes to evaluate their understanding.

Performance Record

This entity stores quiz attempt data such as:

StudentID
QuizID
Score
AttemptDate
Duration

It allows tracking of student learning progress.

Relationships

Authors upload Study Materials and create Quizzes.

Students access, rate, and download Study Materials and attempt Quizzes, generating Performance Records.Admins manage users, moderate content, and supervise system activities.

This interconnected model ensures efficient data management, reliable information exchange, and a smooth learning experience within the SmartStudy platform.

3.7.1 Entity Relationship (ER) Diagram

The ER Diagram illustrates the relationships between the key entities of the SmartStudy system including User, Study Material, Quiz, and Performance Record

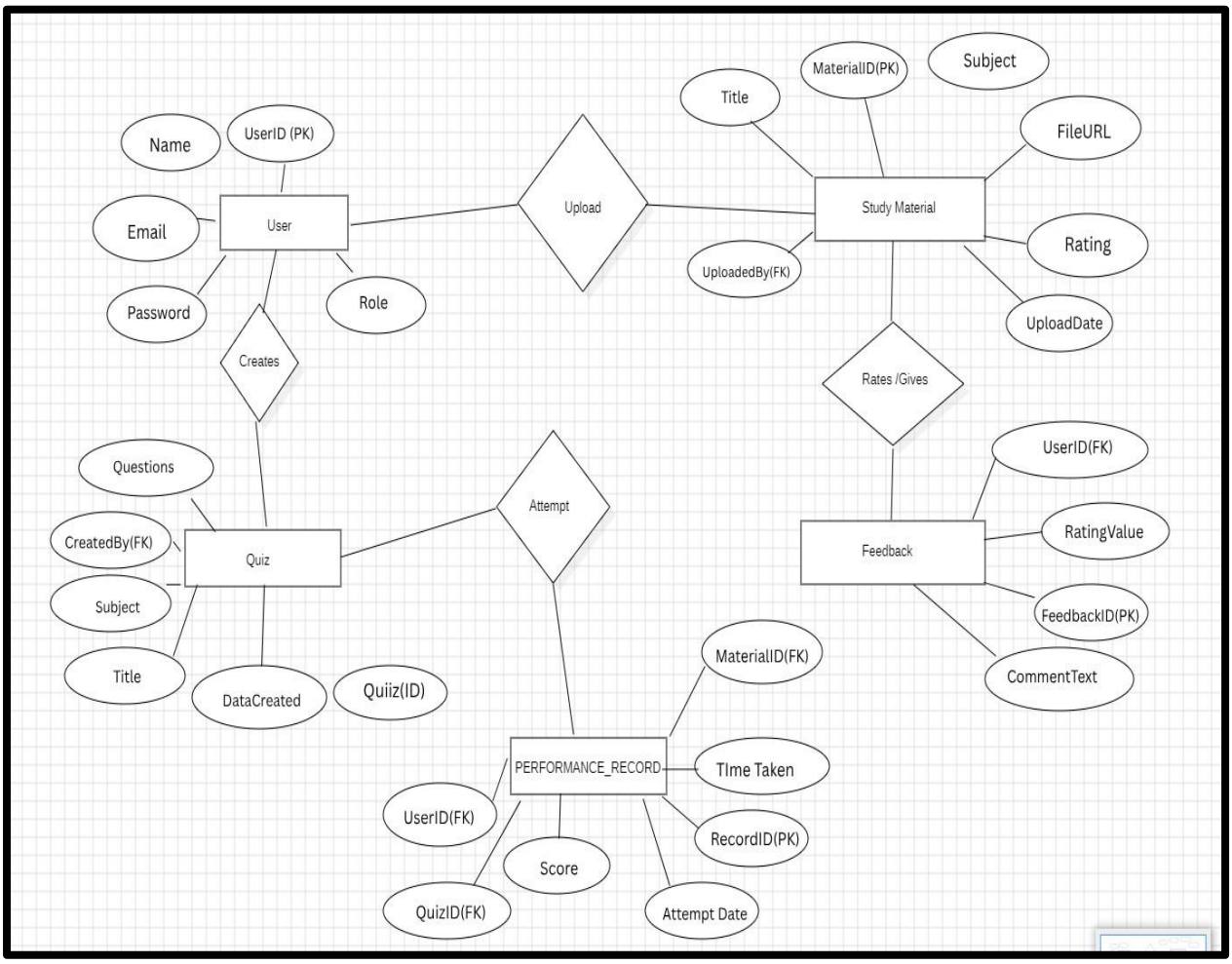
Key Relationships

Author → Content (1:N): A single Author can upload multiple study notes and create various quizzes.

Student → Study Material (N:M): Multiple students can access, download, and rate the same study materials.

Student → Performance (1:N): One student can have many unique quiz attempt records over time.

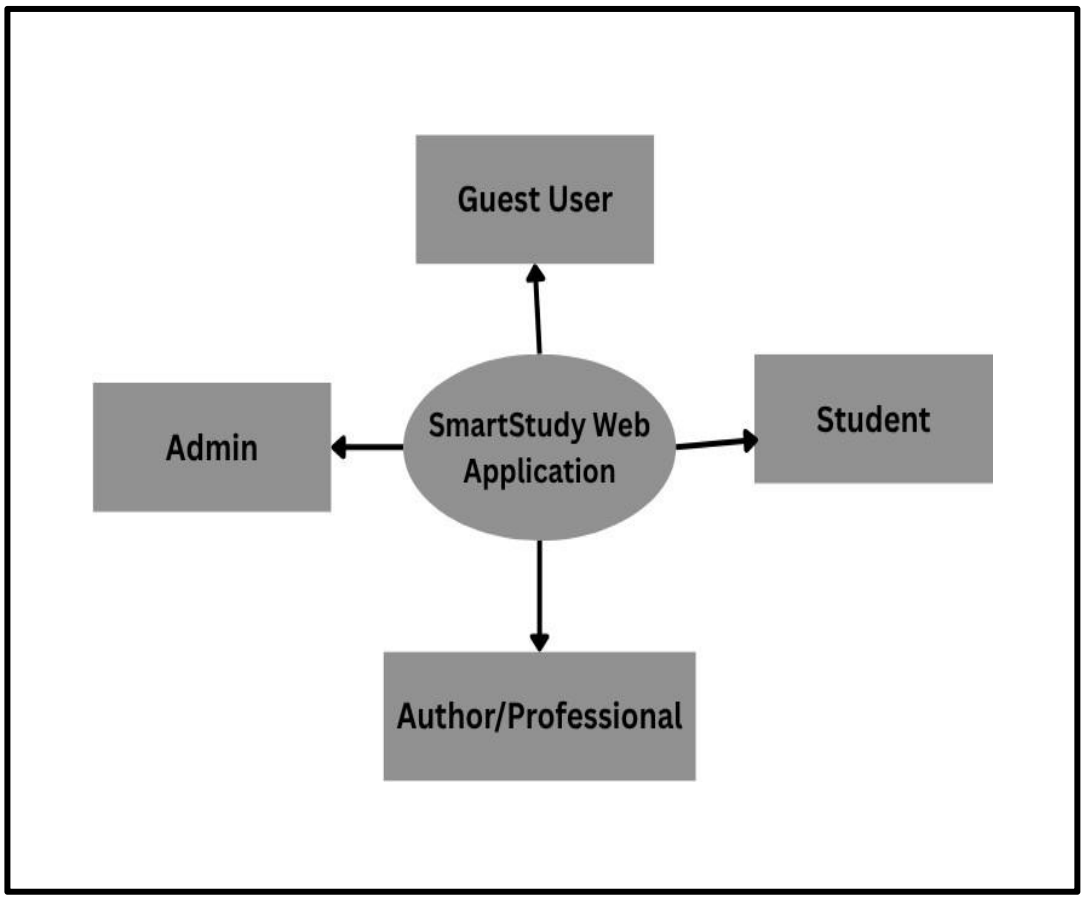
Quiz → Performance (1:N): A specific quiz can be attempted by multiple different students, generating separate records.



3.6.2 Data Flow Diagrams (DFDs) for SmartStudy

1. DFD Level 0: Context Diagram

The **Level 0 Data Flow Diagram** represents the entire SmartStudy system as a single process interacting with external users.



Users such as **Students, Authors, Admins, and Guests** interact with the system to perform actions like registration, login, accessing study materials, attempting quizzes, and managing content. The system processes these requests and stores or retrieves data from the central database implemented using SQLite.

2. DFD Level 1: Core System Modules

The **Level 1 Data Flow Diagram** divides the SmartStudy system into major modules.

Main modules include:

User Management

Study Material Management

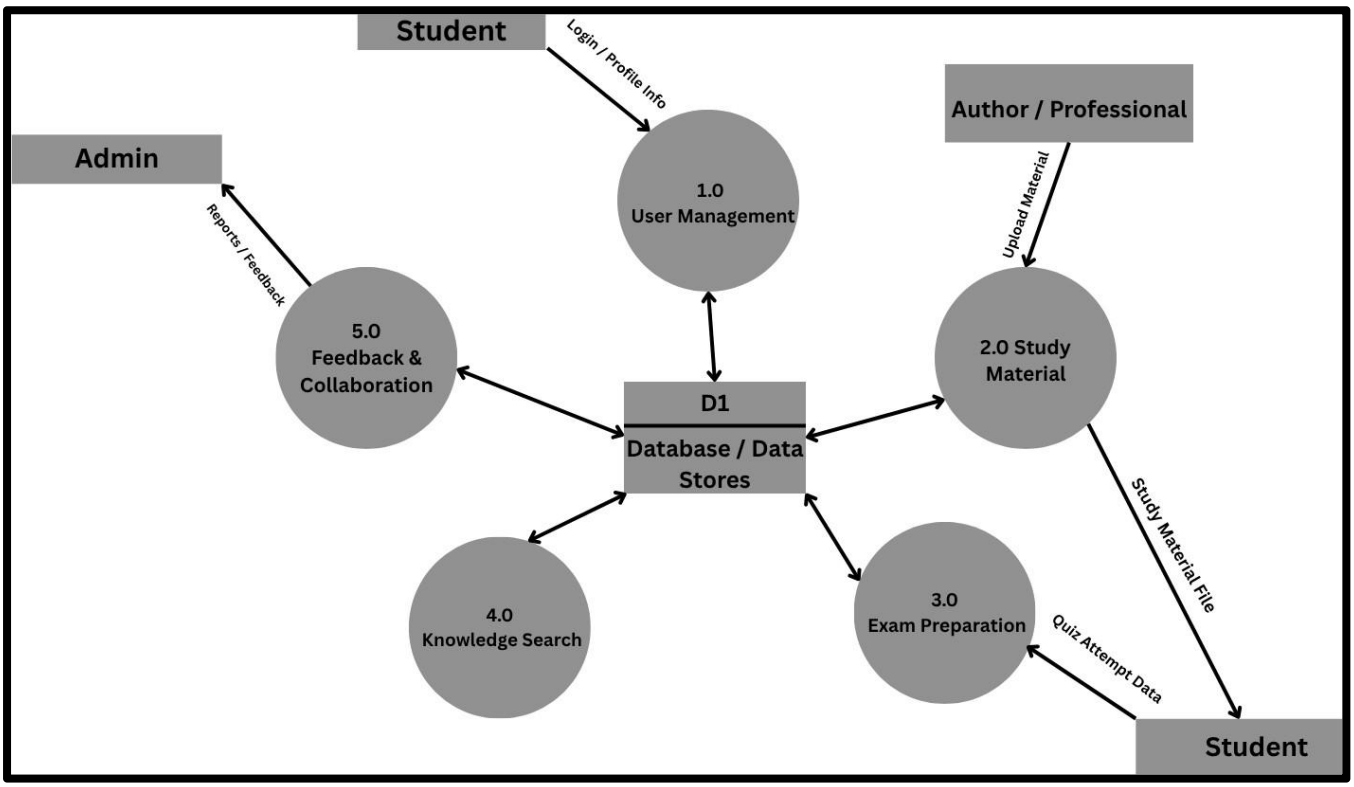
Exam Preparation (Quiz)

Knowledge Search

Feedback and Rating

Admin Management

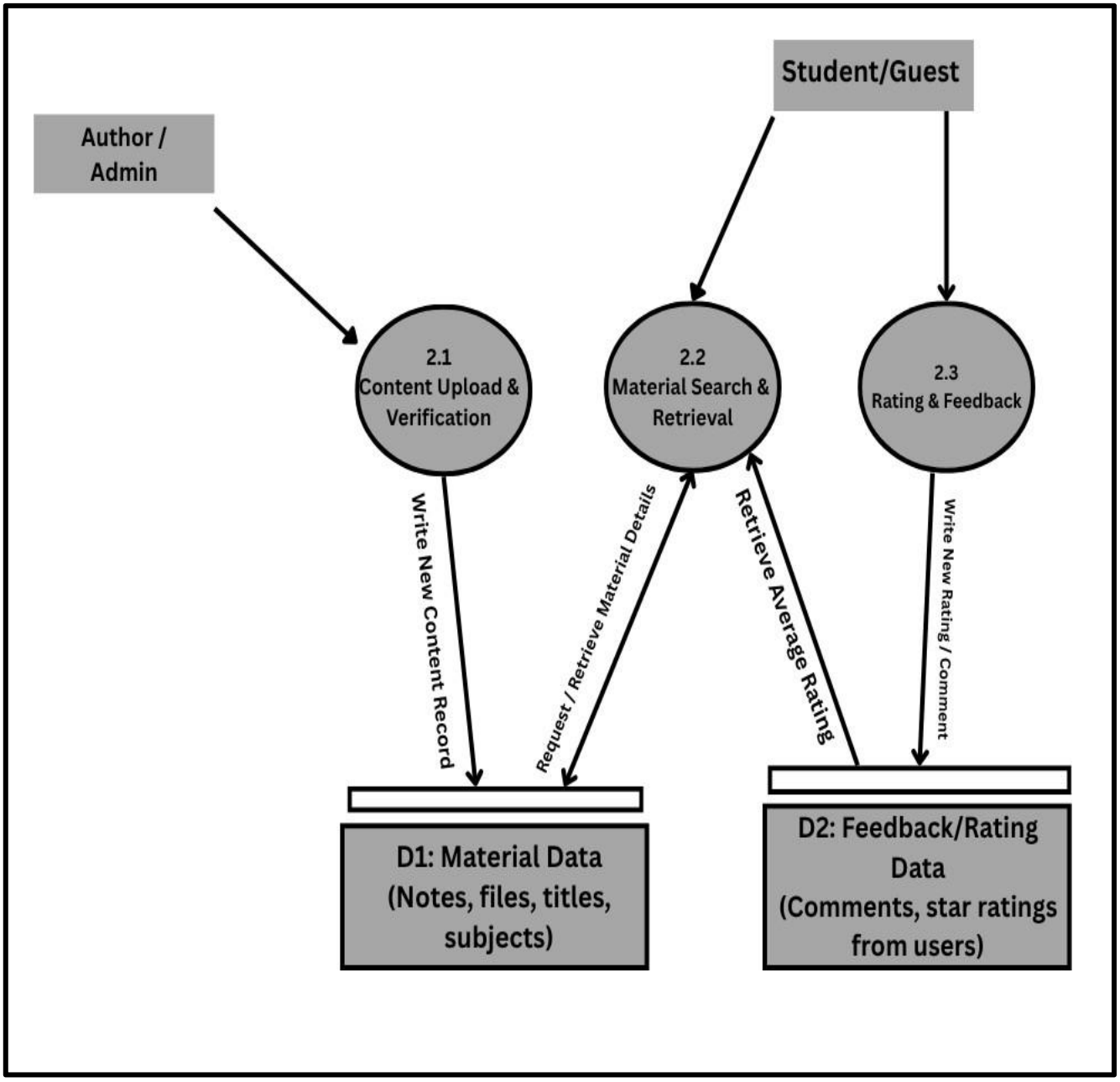
All modules interact with the central database (**D1 – SQLite Database**) to store and retrieve system data.



3. DFD Level 2: Study Material Module

The **Level 2 DFD** provides a detailed view of the Study Material Module.

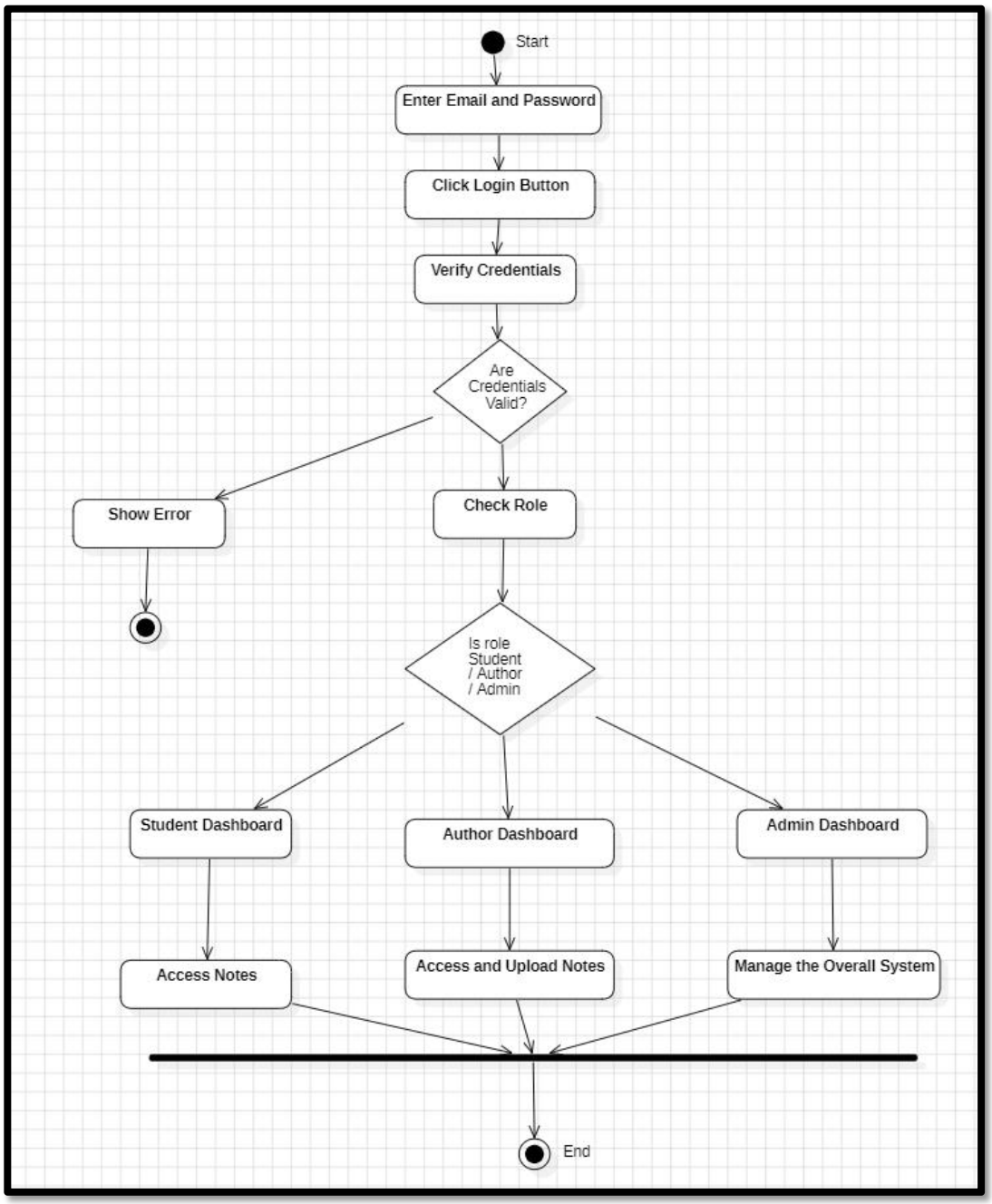
Authors or Admins upload study materials which are verified and stored in the database. Students and guest users can search, view, and download these materials. Students can also rate and provide feedback, which helps improve content quality.



3.6.3 Activity Diagram

The **Activity Diagram** illustrates the workflow of user activities in the SmartStudy system.

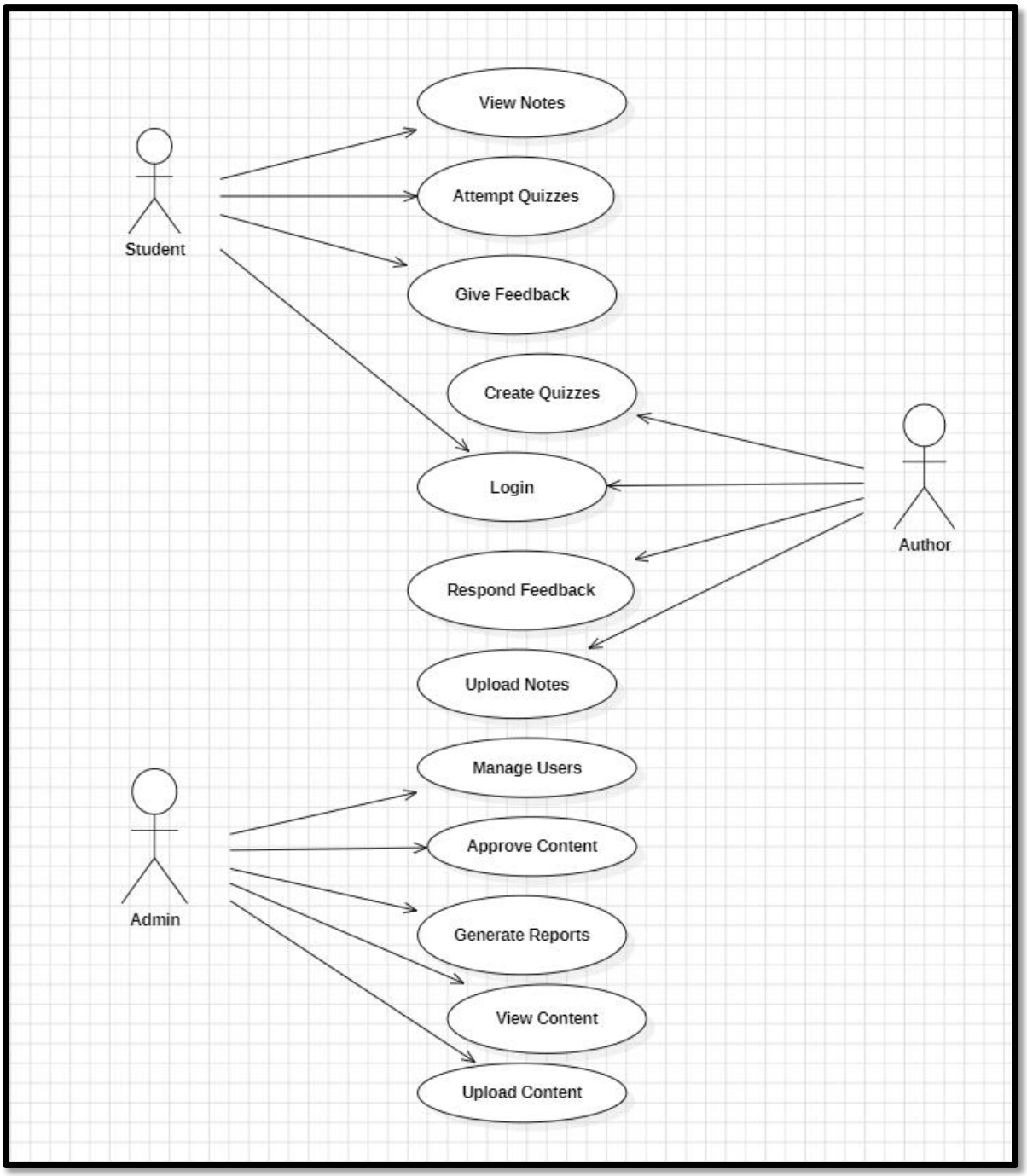
The process begins when a user enters login credentials. After verification, the system identifies the user role (Student, Author, Admin, or Guest) and redirects them to the appropriate dashboard where they can perform their respective tasks.



3.6.4 Use Case Diagram

The **Use Case Diagram** shows how different users interact with the SmartStudy system.

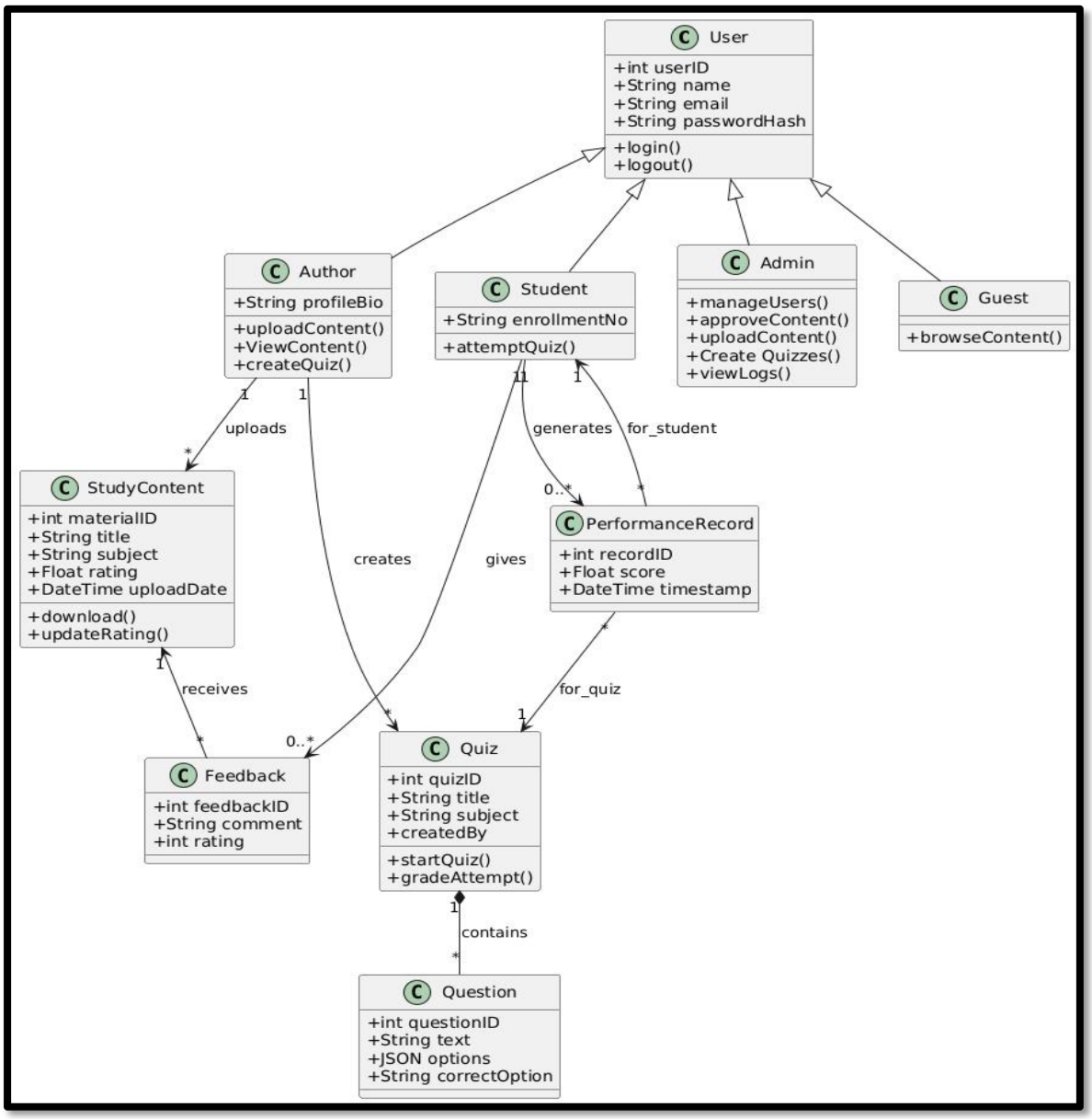
A use case diagram visually represents how different users interact with the system.In *SmartStudy*,It shows how **Students** can register, log in, access study materials, attempt quizzes, and give feedback, while **Professionals** (teachers) can upload and manage notes. The **Admin** oversees user activities and maintains the overall system functionality.



3.6.5 Class Diagram :

The **Class Diagram** represents the structure of the SmartStudy system.

This is a UML **Class Diagram** outlining the structure of the **SmartStudy** application, where specialized **User** roles (**Student, Author, Admin, Guest**) interact with core content (**StudyMaterial, Quiz**) and generate transactional data (**PerformanceRecord, Feedback**).



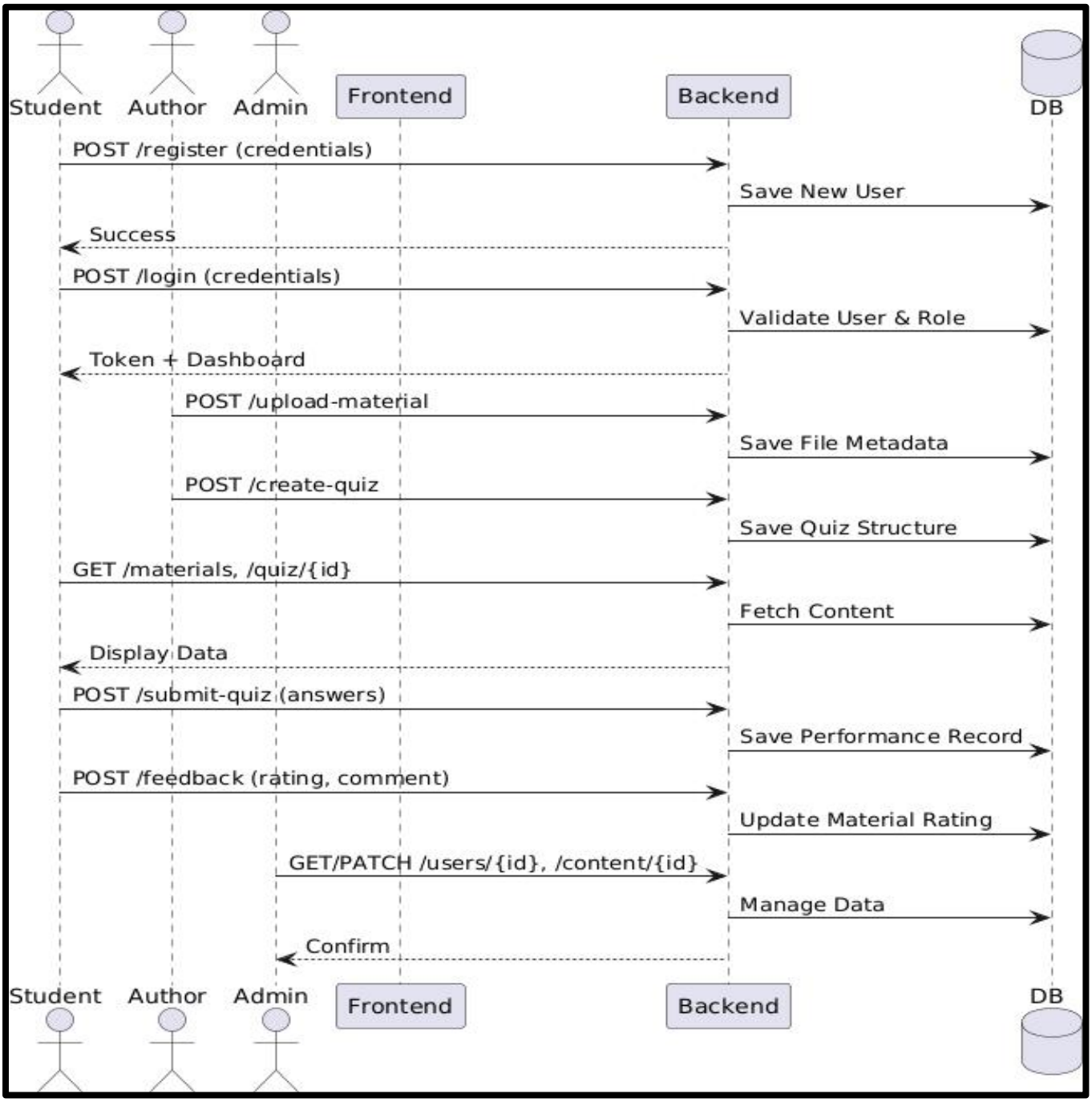
This system is built around a central **User** hierarchy, where specialized roles like **Authors** create and upload **StudyMaterial** and **Quizzes**, and **Students** consume this content, generating metrics stored in **PerformanceRecords** and giving **Feedback**. **Admins** oversee the system and manage all users and content

3.6.6 Sequence Diagram

The **Sequence Diagram** shows the interaction between system components.

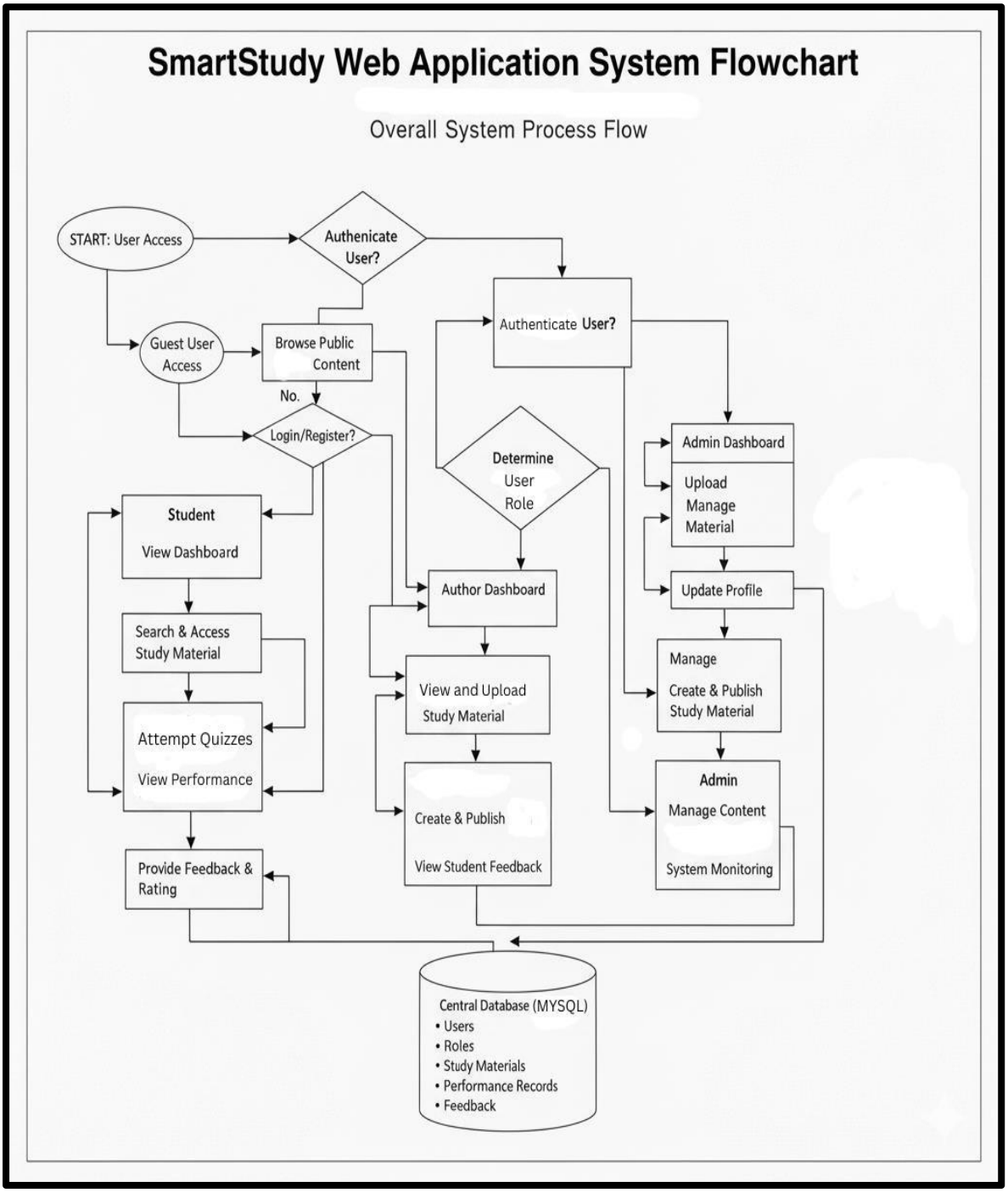
It represents communication between:User (Student / Author / Admin) ,Frontend interface using React ,Backend server using Django ,Database using SQLite

It covers processes like authentication, content upload, quiz attempts, and feedback submission.



3.6.7 System Flowchart

The **System Flowchart** explains the overall working of the SmartStudy web application.The process starts with **user authentication**, after which the system identifies the user role. Based on the role, users can access specific features such as viewing materials, uploading content, attempting quizzes, or managing the platform. All activities interact with the central database implemented using SQLite.



CHAPTER 4: SYSTEM DESIGN

4.1 Overview

The system design of SmartStudy is structured as a modular, full-stack web application aimed at providing a robust, interactive, and secure learning platform for Mumbai University IT/CS students. The architecture emphasizes a clear separation of concerns between the frontend, backend, API services, and database management using SQLite, ensuring maintainability, scalability, and a responsive user experience. This chapter details the system's modular design, data structures, UI specifications, security considerations, and testing strategy.

4.2 Module Breakdown

The system is divided into **six primary modules**, each handling distinct responsibilities.

4.2.1 User Management Module

User Registration/Login:
Handles account creation, login, and password management.

Role-Based Access:
Differentiates between Students, Authors/Professionals, Guest users, and Admin users.

Profile Management:
Allows students and authors to update personal details and preferences.

Session Management:
Maintains secure user sessions and authentication tokens.

4.2.2 Study Material Module

Content Upload:
Authors can upload notes, tutorials, and study resources in a syllabus-aligned manner.

Content Organization:
Categorizes study material by course, semester, and topic.

Download/Access Control:

Students can access or download resources based on permissions.

Rating and Feedback:

Students can rate materials and provide comments for quality assessment.

4.2.3 Exam Preparation Module

Quiz Creation:

Authors can create topic-wise quizzes and MCQs.

Quiz Attempt:

Students can attempt quizzes with time limits and receive instant feedback.

Performance Tracking:

Stores quiz scores in the SQLite database and generates reports for self-assessment.

Progress Analysis:

Students can track improvement over time.

4.2.4 Knowledge Search Module

Search Engine:

Allows topic-based and keyword-driven content searches.

Filtering:

Supports filters for course, semester, topic, and content type.

Recommendations:

Suggests related study materials based on search queries or student activity.

4.2.5 Feedback and Collaboration Module

Student Feedback:

Enables rating and commenting on notes and quizzes.

Author Interaction:

Authors can respond to student queries or comments.

Peer Sharing:

Students can share study tips or resources within the platform.

4.2.6 Admin Management Module

The Admin module is responsible for **system monitoring, content moderation, and user management** to maintain platform quality and security.

User Management:

Admin can view, approve, suspend, or remove student and author accounts.

Content Moderation:

Admin can review uploaded notes, quizzes, and study materials to ensure they follow academic guidelines.

System Monitoring:

Admin can monitor platform activity such as number of users, uploaded materials, and quiz attempts.

Feedback Management:

Admin can review reported content and resolve disputes or inappropriate feedback.

Data Management:

Admin ensures that the system database using SQLite remains consistent and secure.

4.3 Data Design

The data design defines the structure, constraints, and relationships of key entities in SmartStudy. All system data is stored and managed using the lightweight relational database system SQLite integrated with the backend framework.

4.3.1 User Data

User data includes student, author, and admin account details such as name, email, password, role, and profile information. Each user is uniquely identified by a **UserID**, which acts as the primary key in the database.

This data enables authentication, authorization, and personalized access to study materials and quizzes.

4.3.2 Study Material Data

Field Name	Type	Description
MaterialID	Integer	Primary Key
Title	String	Name of the material
Subject	String	Course/Subject category
UploadedBy	Integer	Foreign Key → UserID
FileURL	String	Path to uploaded file
Rating	Float	Average user rating
UploadDate	DateTime	Date of upload

This data is stored in the SQLite database and accessed using ORM queries through the backend framework.

4.3.3 Quiz Data

Field Name	Type	Description
QuizID	Integer	Primary Key
Title	String	Quiz title

Field Name	Type	Description
Subject	String	Related course/subject
CreatedBy	Integer	Foreign Key → UserID
Questions	JSON	Stores MCQs and options
DateCreated	DateTime	Timestamp of quiz creation

The quiz data structure allows dynamic quiz creation and storage within the SQLite database.

4.4 UI Design

The user interface is designed to be intuitive, responsive, and student-friendly, supporting both desktop and mobile devices.

4.4.1 Dashboard Layout

Student Dashboard:
Shows personalized notes, quizzes, progress, and recent activity.

Author Dashboard:
Enables content upload, quiz creation, and review of student feedback.

Admin Dashboard:
Allows administrators to manage users, review content uploads, monitor system activities, and maintain platform security.

Guest View:
Provides limited access to search and preview materials.

4.4.2 Navigation and Interaction

Sidebar/Menu:
Quick links to notes, quizzes, search, feedback, and admin management options.

Content Display:

Grid or list view for study materials with ratings and download buttons.

Interactive Quizzes:

Displays MCQs, timers, and instant feedback upon submission.

4.5 Security Considerations

Authentication & Authorization:

Secure login system with role-based access control.

Data Privacy:

Student, author, and admin information is encrypted and access-controlled.

Input Validation:

Prevents malicious file uploads and invalid data entries.

Secure API Communication:

Uses HTTPS and token-based authentication for backend requests.

Database Security:

Data stored in the SQLite database is protected using ORM queries, preventing SQL injection and unauthorized access.

4.6 Test Case Design

4.6.1 User Module Test Cases

Test Case ID	Description	Input	Expected Result
TC-UM-01	Student Registration	Valid Name, Email, Password	Success: New Student Account Created

Test Case ID	Description	Input	Expected Result
TC-UM-02	Author Login	Valid Credentials	Success: Redirect to Author Dashboard
TC-UM-03	Invalid Login Attempt	Wrong Password/Email	Error Message: Invalid Credentials

4.6.2 Study Material Module Test Cases

Test Case ID	Description	Input	Expected Result
TC-SM-01	Upload Valid Material	PDF/Doc File	Success: Material Uploaded
TC-SM-02	Upload Invalid File Format	.exe / .txt	Error: Invalid File Type
TC-SM-03	Rate Material	Rating Value 1–5	Average Rating Updated
TC-SM-04	Download Study Material	Valid Student Request	Success: File Downloaded / Access Granted

4.6.3 Quiz Module Test Cases

Test Case ID	Description	Input	Expected Result

Test Case ID	Description	Input	Expected Result
TC-QZ-01	Attempt Quiz	Valid Answers	Score Calculated, Feedback Displayed
TC-QZ-02	Time Limit Exceeded	Not Completed	Quiz Submission Closed, Score Recorded
TC-QZ-03	Invalid Question Format	Corrupt JSON	Error: Quiz Cannot Be Loaded

4.7 Conclusion

The SmartStudy system design establishes a modular and scalable architecture for a web-based learning platform. Each module — **User Management, Study Materials, Quizzes, Search, Feedback, and Admin Management** — is clearly defined to improve maintainability and usability. The data design ensures secure and organized storage of user information, study content, and quiz records within the SQLite database. Security mechanisms and test case planning guarantee system reliability and robustness. Overall, this design provides a solid foundation for implementation, deployment, and future expansion of SmartStudy.

CHAPTER 5 : IMPLEMENTATION AND TESTING

Home.jsx :

```
import Navbar from "../components/Navbar";
import Hero from "../components/Hero";
import Footer from "../components/Footer";
import { useNavigate } from "react-router-dom";
import { useState, useEffect } from "react";
export default function Home() {
const navigate = useNavigate();
const [searchTerm, setSearchTerm] = useState("");
const [subjects, setSubjects] = useState([]);
const [suggestions, setSuggestions] = useState([]);
// Fetch subjects
useEffect(() => {
fetch("http://127.0.0.1:8000/api/subjects/")
.then(res => res.json())
.then(setSubjects)
.catch(console.error);
}, []);
// Hover effects (reusable)
const handleHover = (e, scale, shadow) => {
e.currentTarget.style.transform = `scale(${scale})`;
e.currentTarget.style.boxShadow = shadow;
};
// Explore button
const handleExploreClick = () => {
const user = localStorage.getItem("user");
navigate(user ? "/explore-certification" : "/login", {
state: !user && {
message:
"To complete the course and quizzes for certification, please login or sign up.",
},
});
};
// Input change + suggestions
const handleChange = (e) => {
const value = e.target.value;
setSearchTerm(value);
setSuggestions(
value
? subjects.filter(s =>
s.title.toLowerCase().includes(value.toLowerCase())
)
: []
);
};
// Suggestion click
const handleSuggestionClick = (id, title) => {
setSearchTerm(title);
setSuggestions([]);
```

```
navigate(`/study-material/${id}`);
};
// Search logic
const handleSearch = () => {
if (!searchTerm.trim()) return;
const text = searchTerm.toLowerCase();
const year = text.match(/\b20\d{2}\b/)?.[0] || "";
if (/(pyq|previous year|question)/.test(text)) {
const subject = text
.replace(/pyq|previous year|question|\b20\d{2}\b/g, "")
.trim();
return navigate(
`/previous-year-questions?subject=${subject}&year=${year}`
);
}
navigate(`/study-material?search=${searchTerm}`);
};
return (
<div style={styles.page}>
<Navbar />
<Hero />
{/* Search */}
<section style={styles.searchSection}>
<h2 style={styles.searchTitle}>Let's Start Learning</h2>
<div style={styles.searchBox}>
<div style={styles.searchWrapper}>
<input
type="text"
placeholder="Search study materials, PYQs..."
style={styles.searchInput}
value={searchTerm}
onChange={handleChange}
/>
<button style={styles.searchButton} onClick={handleSearch}>
Search
</button>
</div>
{suggestions.length > 0 && (
<div style={styles.suggestionBox}>
{suggestions.map(item => (
<div
key={item.id}
style={styles.suggestionItem}
onClick={() =>
handleSuggestionClick(item.id, item.title)
}
>
{item.title}
</div>
))}
</div>
)}
</div>
</section>
{/* About */}
```

```
<section style={styles.aboutSection}>
<div style={styles.cardsContainer}>
{[
["Purpose", "SmartStudy simplifies learning."],
["Syllabus Notes", "Mumbai University aligned notes."],
["Quizzes", "Topic-wise quizzes."],
["Collaboration", "Students & authors interact."],
].map(([title, text], i) => (
<div
key={i}
style={styles.card}
onMouseEnter={(e) =>
handleHover(e, 1.04, "0 20px 40px rgba(0,0,0,0.08)")
}
onMouseLeave={(e) =>
handleHover(e, 1, "0 10px 25px rgba(0,0,0,0.05)")
}
>
<h3 style={styles.cardTitle}>{title}</h3>
<p style={styles.cardText}>{text}</p>
</div>
))}
</div>
</section>
{/* Certification */}
<section style={styles.certSection}>
<div style={styles.certBox}>
<h2 style={styles.certTitle}>
Get Certified in Your Preferred Topics
</h2>
<button
style={styles.certButton}
onClick={handleExploreClick}
>
Click to Explore
</button></div>
</section><Footer /></div>

);}
```

App.jsx :

```
import { Routes, Route } from "react-router-dom";

import Home from "./pages/Home";

import Blog from "./pages/Blog";

import Connect from "./pages/Connect";

import Reference from "./pages/Reference";

import Login from "./pages/Login";
```

```
import Signup from "./pages/Signup";
import StudentDashboard from "./pages/StudentDashboard";
import HomeStudyMaterial from "./pages/HomeStudyMaterial";
import AuthorDashboard from "./pages/AuthorDashboard";
import PreviousYearQuestions from "./pages/PreviousYearQuestions";
import TopicContent from "./pages/TopicContent";
import CertificationQuiz from "./pages/CertificationQuiz";
import ExploreCertification from "./pages/ExploreCertification";
import CertificationResults from "./pages/CertificationResults";
import CertificationCourse from "./pages/CertificationCourse";
import QuizPage from "./pages/QuizPage";
import CertificationSuccess from "./pages/CertificationSuccess";
import CertificationPreview from "./pages/CertificatePreview"; //     filename check karo
function App() {
return (
<Routes>
{/* CERTIFICATION FLOW */}
<Route path="/certification-quiz/:id" element={<CertificationQuiz />} /> {/*     ADD THIS */}
<Route path="/explore-certification" element={<ExploreCertification />} />
<Route path="/certifications" element={<CertificationResults />} /> {/*     ADD THIS */}
<Route path="/certification/:id" element={<CertificationCourse />} />
<Route path="/quiz/:id" element={<QuizPage />} />
<Route path="/certification/:id/success" element={<CertificationSuccess />} />
<Route path="/certificate/:id" element={<CertificationPreview />} />
{/* STUDY MATERIAL */}
<Route path="/study-material/:id" element={<TopicContent />} />
```

```
<Route path="/study-material" element={<HomeStudyMaterial />} />
{/* OTHER ROUTES */}
<Route path="/previous-year-questions" element={<PreviousYearQuestions />} />
<Route path="/author-dashboard" element={<AuthorDashboard />} />
<Route path="/student-dashboard" element={<StudentDashboard />} />
<Route path="/" element={<Home />} />
<Route path="/blog" element={<Blog />} />
<Route path="/connect" element={<Connect />} />
<Route path="/reference" element={<Reference />} />
<Route path="/login" element={<Login />} />
<Route path="/signup" element={<Signup />} />
</Routes>
); }
 export default App;
```

Login.jsx:

```
import { useState } from "react";
import { useNavigate, Link, useLocation } from "react-router-dom";
export default function Login() {
  const navigate = useNavigate();
  const { state } = useLocation();
  const [form, setForm] = useState({ username: "", password: "" });
  const handleLogin = async () => {
    try {
      const res = await fetch("http://127.0.0.1:8000/api/login/", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify(form),
    });
    const data = await res.json();
    if (!res.ok) return alert(data.error || "Invalid credentials ✕");
    localStorage.setItem("user", data.username);
    localStorage.setItem("role", data.role);

    alert("Login Successful ✅");
    navigate(
      data.role === "student"
        ? "/student-dashboard"
        : data.role === "author"
        ? "/author-dashboard"
        : "/"    );  }
catch {
    alert("Server not reachable ✕");
  }  };
 return (
   <div>
    <button onClick={() => navigate("/")}>← Go Back</button>
    <h2>Login</h2>
    {state?.message && <p>{state.message}</p>}
    <input
      placeholder="Username"
      value={form.username}
      onChange={(e) => setForm({ ...form, username: e.target.value })}
```

```
    />
    <input
      type="password"
      placeholder="Password"
      value={form.password}
      onChange={(e) => setForm({ ...form, password: e.target.value })}
    />
    <button onClick={handleLogin}>Login</button>
    <p>
      Don’t have an account? <Link to="/signup">Sign up</Link>
    </p></div> );}
```

Signup.jsx:

```
import { useNavigate, Link } from "react-router-dom";
import { useState } from "react";
export default function Signup() {
  const navigate = useNavigate();
  const [form, setForm] = useState({
    username: "",
    email: "",
    password: "",
    confirmPassword: "",
    role: ""
  });
  const handleSignup = async () => {
    const { username, email, password, confirmPassword, role } = form;
    if (password !== confirmPassword)
```

```
    return alert("Passwords do not match ❌");
  if (!role)
    return alert("Please select a role ❌");
  try {
    const res = await fetch("http://127.0.0.1:8000/api/signup/", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username, email, password, role }),
    });const data = await res.json();
    if (!res.ok)
      return alert(data.error || "User already exists ❌");
    localStorage.setItem("user", data.username);
    localStorage.setItem("role", data.role);
    alert("Account created successfully ✅");
    navigate(
      data.role === "student"
        ? "/student-dashboard"
        : data.role === "author"
        ? "/author-dashboard"
        : "/"  );
  } catch {
    alert("Server not reachable ❌");
  } };
const handleChange = (e) =>
  setForm({ ...form, [e.target.name]: e.target.value });
return (  <div>
```

```
    <button onClick={() => navigate("/")}>← Go Back</button>

    <h2>Sign Up</h2>

    <input name="username" placeholder="Username" onChange={handleChange} />

    <input name="email" type="email" placeholder="Email" onChange={handleChange} />

    <input name="password" type="password" placeholder="Password" onChange={handleChange} />

    <input name="confirmPassword" type="password" placeholder="Confirm Password"
onChange={handleChange} />

    <select name="role" onChange={handleChange}>

      <option value="">Select Role</option>

      <option value="student">Student</option>

      <option value="author">Author</option></select>

    <button onClick={handleSignup}>Create Account</button>  <p>

      Already have an account? <Link to="/login">Login</Link>

    </p></div>  );}
```

StudentDashboard.jsx :

```
import { useEffect, useState } from "react";

import { useNavigate } from "react-router-dom";

import { PieChart, Pie, Cell, Tooltip, Legend, ResponsiveContainer } from "recharts";

export default function StudentDashboard() {

  const navigate = useNavigate();

  const user = localStorage.getItem("user");

  const role = localStorage.getItem("role");

  const [activeSection, setActiveSection] = useState("home");

  const [searchTerm, setSearchTerm] = useState("");

  const [subjects, setSubjects] = useState([]);

  const [suggestions, setSuggestions] = useState([]);

  const [viewedTopics, setViewedTopics] = useState([]);
```

```
const [attemptedQuizzes, setAttemptedQuizzes] = useState([]);
const [analytics, setAnalytics] = useState(null);
// Protect route
useEffect(() => {
  if (!user) return navigate("/login");
  if (role !== "student") navigate("/");
}, [user, role, navigate]);
// Fetch subjects
useEffect(() => {
  fetch("http://127.0.0.1:8000/api/subjects/")
    .then(res => res.json())
    .then(setSubjects)
    .catch(console.error);
}, []);
// Reusable fetch
const fetchData = (url, setter) => {
  fetch(url)
    .then(res => res.json())
    .then(setter)
    .catch(console.error);
};
const handleLogout = () => {
  localStorage.clear();
  navigate("/");
};
// Search + suggestions
```

```
const handleChange = (e) => {
  const value = e.target.value;
  setSearchTerm(value);
  setSuggestions(
    value
      ? subjects.filter(s =>
          s.title.toLowerCase().includes(value.toLowerCase())
        )  : []  ); };
const handleSuggestionClick = (id, title) => {
  setSearchTerm(title);
  setSuggestions([]);
  navigate(`/study-material/${id}`);
};
const handleSearch = () => {
  if (!searchTerm.trim()) return;
  const text = searchTerm.toLowerCase();
  const year = text.match(/\b20\d{2}\b/)?.[0] || "";
  if (/(pyq|previous year|question)/.test(text)) {
    const subject = text.replace(/pyq|previous year|question|\b20\d{2}\b/g, "").trim();
    return navigate(`/previous-year-questions?subject=${subject}&year=${year}`);
  }
  navigate(`/study-material?search=${searchTerm}`);
};
if (!user || role !== "student") return null;
const chartData = analytics
  ? [
```

```
      { name: "Topics Viewed", value: analytics.topics_viewed },
      { name: "Quizzes Attempted", value: analytics.quizzes_attempted },
      { name: "Certifications", value: analytics.certifications }
    ]
  : [];
const COLORS = ["#4f46e5", "#06b6d4", "#22c55e"];
return (
  <div style={styles.page}>
    {/* HEADER */}
    <div style={styles.hero}>
      <div>
        <h1>Student Dashboard</h1>
        <p>SmartStudy</p>
      </div>
      <button onClick={() => navigate("/")}>Home</button>
    </div>
    <div style={styles.wrapper}>
      {/* SIDEBAR */}
      <div style={styles.sidebar}>
        <div style={styles.profile}>
          <div style={styles.avatar}>{user[0].toUpperCase()}</div>
          <h3>{user}</h3>
          <p>Student</p> </div>
        {[
          ["Viewed Topics", "viewed", () =>
            fetchData(`http://127.0.0.1:8000/api/viewed-topics/${user}/`, setViewedTopics)
```

```
      ],
      ["Attempted Quizzes", "quiz", () =>
        fetchData(`http://127.0.0.1:8000/api/attempted-quizzes/${user}/`, setAttemptedQuizzes)
      ],
      ["Certifications", "cert"],
      ["Leaderboard", "leaderboard", () =>
        fetchData(`http://127.0.0.1:8000/api/student-analytics/${user}/`, setAnalytics)
      ],
    ].map(([label, key, fn]) => (
      <button
        key={key}
        style={styles.menuBtn}
        onClick={() => {
          setActiveSection(key);
          fn && fn();
        }}
      >
        {label}
      </button>
    ))}
    <button style={styles.logoutBtn} onClick={handleLogout}>
      Logout
    </button>
  </div>
  {/* MAIN */}
  <div style={styles.main}>
```

```
{/* SEARCH */}
<div style={styles.searchBox}>
  <div style={styles.search}>
    <input
      placeholder="Search..."
      value={searchTerm}
      onChange={handleChange}   />
    <button onClick={handleSearch}>Search</button>
  </div>
  {suggestions.length > 0 && (
    <div style={styles.suggestions}>
      {suggestions.map(s => (
        <div
          key={s.id}
          onClick={() => handleSuggestionClick(s.id, s.title)}   >
          {s.title}
        </div> ))}</div> )}</div>
{/* CONTENT */}
<div>
  {/* Viewed */}
  {activeSection === "viewed" && (<>
      <h2>Viewed Topics</h2>
      <div style={styles.grid}>
        {!viewedTopics.length && <p>No data</p>}
        {viewedTopics.map((t, i) => (
          <div key={i} onClick={() => navigate(`/study-material/${t.subject_id}`)}>
```

```
            <h4>{t.title}</h4> <p>{new Date(t.viewed_at).toLocaleDateString()}</p>
        </div>  ))}   </div>  </>    )}
    {/* Quiz */}
    {activeSection === "quiz" && (<>
      <h2>Attempted Quizzes</h2>
      <div style={styles.grid}>
        {!attemptedQuizzes.length && <p>No data</p>}
        {attemptedQuizzes.map((q, i) => (
          <div key={i} onClick={() => navigate(`/study-material/${q.subject_id}`)}>
            <h4>{q.quiz_title}</h4>
            <p>{q.subject_title}</p>
            <p>{q.score}/{q.total}</p>    </div>    ))}  </div>
      </>  )}
    {/* Analytics */}
    {activeSection === "leaderboard" && analytics && (
     <>
      <h2>Analytics</h2>
      <ResponsiveContainer width="100%" height={300}>
        <PieChart>
          <Pie data={chartData} dataKey="value" outerRadius={100} label>
            {chartData.map((_, i) => (
              <Cell key={i} fill={COLORS[i % COLORS.length]} />
            ))}
          </Pie>
          <Tooltip />
          <Legend />
```

```
            </PieChart>
          </ResponsiveContainer>  </>  )}
</div>  </div> </div> </div>  );}
```

AuthorDashboard.jsx:

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import Profile from "../components/author/AuthorProfile";
import Upload from "../components/author/UploadMaterial";
import Quiz from "../components/author/CreateQuiz";
import Leaderboard from "../components/author/Leaderboard";
import Notifications from "../components/author/Notifications";
export default function AuthorDashboard() {
  const navigate = useNavigate();
  const user = localStorage.getItem("user");
  const role = localStorage.getItem("role");
  const [active, setActive] = useState("profile");
  useEffect(() => {
    if (!user) return navigate("/login");
    if (role !== "author") navigate("/");
  }, [user, role, navigate]);
  const handleLogout = () => {
    localStorage.clear();
    navigate("/");
  };
  const sections = {
    profile: <Profile />,
```

```
    upload: <Upload />,
    quiz: <Quiz />,
    leaderboard: <Leaderboard />,
    notifications: <Notifications />,};
  if (!user || role !== "author") return null;
  return (
    <div>
      {/* HEADER */}
      <div>
        <div>  <h1>Author Dashboard</h1>
          <p>SmartStudy</p>
        </div>
        <button onClick={() => navigate("/")}>Home</button>
      </div>
      <div style={{ display: "flex" }}>
        {/* SIDEBAR */}
        <div>
          <div>
            <div>{user[0].toUpperCase()}</div>
            <h3>{user}</h3>
            <p>Author</p>
          </div>
          {[
            ["Profile", "profile"],
            ["Upload Material", "upload"],
            ["Create Quiz", "quiz"],
```

```
            ["Leaderboard", "leaderboard"],
            ["Notifications", "notifications"],
          ].map(([label, key]) => (
            <button key={key} onClick={() => setActive(key)}>
              {label}
            </button>
          ))}
          <button onClick={handleLogout}>Logout</button>
        </div>
        {/* MAIN */}
        <div>
          {sections[active] || <Profile />}   </div>
      </div>    </div>  );}
```

ExploreCertification.jsx:

```
import { useNavigate } from "react-router-dom";
import { useState } from "react";
export default function ExploreCertification() {
  const navigate = useNavigate();
  const user = localStorage.getItem("user");
  const [search, setSearch] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    if (!search.trim()) return;
    navigate(`/certifications?search=${search}`); };
  return (   <div>
      <h2>Hello, {user}    <br />What are you opting to learn today?</h2>
```

```
    <form onSubmit={handleSubmit}>
      <input
        placeholder="Search Certification Course..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}  />
      <button>Explore Course</button>
    </form>   </div>  );}
```

PreviousYearQuestions.jsx:

```
import { useState, useEffect } from "react";
import { useLocation } from "react-router-dom";
import Navbar from "../components/Navbar";
export default function PreviousYearQuestions() {
  const location = useLocation();
  const [courses, setCourses] = useState([]);
  const [semesters, setSemesters] = useState([]);
  const [years, setYears] = useState([]);
  const [subjects, setSubjects] = useState([]);
  const [course, setCourse] = useState("");
  const [semester, setSemester] = useState("");
  const [subject, setSubject] = useState("");
  const [year, setYear] = useState("");
  const [showPDF, setShowPDF] = useState(false);
  const [pdfUrl, setPdfUrl] = useState("");
  const [pyqCards, setPyqCards] = useState([]);
  // dropdown options
  useEffect(() => {
```

```
  fetch("http://127.0.0.1:8000/api/pyq-options/")
    .then((res) => res.json())
    .then((data) => {
      setCourses(data.courses || []);
      setSemesters(data.semesters || []);
      setYears(data.years || []);
    });
}, []);
// subjects
useEffect(() => {
  if (!course || !semester) return;
  fetch(`http://127.0.0.1:8000/api/pyq-subjects/?course=${course}&semester=${semester}`)
    .then((res) => res.json())
    .then(setSubjects);
}, [course, semester]);
// HOME SEARCH via URL params
useEffect(() => {
  const params = new URLSearchParams(location.search);
  const subjectParam = params.get("subject");
  const yearParam = params.get("year");
  if (!subjectParam && !yearParam) return;
  let url = "http://127.0.0.1:8000/api/pyqs/?";
  if (subjectParam) url += `subject=${subjectParam}&`;
  if (yearParam) url += `year=${yearParam}`;
  fetch(url)
    .then((res) => res.json())
```

```
      .then((data) => data && setPyqCards(data));
  }, [location.search]);
  const handleGetPYQ = async () => {
    if (!course || !semester || !subject || !year) return alert("Please select all fields.");
    try {
      const res = await fetch(
`http://127.0.0.1:8000/api/pyqs/?course=${course}&semester=${semester}&subject=${subject}&year=${year}`
      );
      const data = await res.json();
      if (!data || data.length === 0) return alert("No PYQ found");
      setPyqCards(data);
    } catch {
      alert("Error fetching PYQ");   } };
  const handleBack = () => {
    setShowPDF(false);
    setPdfUrl("");
  };
  const handleDownload = async (pdf, sub, yr) => {
    try {
      const res = await fetch("http://127.0.0.1:8000" + pdf);
      const blob = await res.blob();
      const url = window.URL.createObjectURL(blob);
      const link = document.createElement("a");
      link.href = url;
      link.download = `${sub}-${yr}.pdf`;
```

```
      document.body.appendChild(link);

      link.click();

      link.remove();

      window.URL.revokeObjectURL(url);

    } catch {

      console.error("Download error");

    }

  };

  return (

    <div>

      <Navbar />

      <h2>Previous Year Questions</h2>

      {/* FORM */}

      {pyqCards.length === 0 && !showPDF && (

        <div>

          <div>

            <label>Course</label>

            <select value={course} onChange={(e) => { setCourse(e.target.value); setSemester(""); setSubject(""); setYear(""); }}>

              <option value="">Select Course</option>

              {courses.map((c) => <option key={c.value} value={c.value}>{c.label}</option>)}

            </select>

          </div>

          <div>

            <label>Semester</label>

            <select value={semester} onChange={(e) => { setSemester(e.target.value); setSubject(""); setYear(""); }} disabled={!course}>
```

```
          <option value="">Select Semester</option>
          {semesters.map((s) => <option key={s.value} value={s.value}>{s.label}</option>)}
        </select>
      </div>
      <div>
        <label>Subject</label>
        <select value={subject} onChange={(e) => setSubject(e.target.value)} disabled={!semester}>
          <option value="">Select Subject</option>
          {subjects.map((s) => <option key={s.value} value={s.value}>{s.label}</option>)}
        </select>
      </div>
      <div>
        <label>Year</label>
        <select value={year} onChange={(e) => setYear(e.target.value)} disabled={!semester}>
          <option value="">Select Year</option>
          {years.map((y) => <option key={y.value} value={y.value}>{y.label}</option>)}
        </select>
      </div>
      <button onClick={handleGetPYQ}>Get PYQ</button>
    </div>
  )}
  {/* PYQ CARDS */}
  {pyqCards.length > 0 && !showPDF && (
    <div>
      {pyqCards.map((p, i) => (
        <div key={i}>
```

```
            <h3>{p.subject}</h3>

            <p>Year {p.year}</p>

            <button onClick={() => window.open("http://127.0.0.1:8000"+p.pdf,"_blank")}>View</button>

            <button onClick={() => handleDownload(p.pdf, p.subject, p.year)}>Download</button>

          </div>   ))}

       </div>

      )}

      {/* PDF VIEWER */}

      {showPDF && (

        <div>

          <iframe src={pdfUrl + "#toolbar=0"} title="PYQ PDF" style={{ width: "100%", height: "500px",
border: "none" }} />

          <button onClick={handleBack}>Back</button>   </div>)}</div> );}
```

Settings.py :

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-iyrgqmcwp7e9ex+kaqz+k(vgfm73--l-t+)_(u@b5=g68f23h8'

DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = [

    'rest_framework',

    'corsheaders',

    'django.contrib.admin',

    'django.contrib.auth',

    'django.contrib.contenttypes',

    'django.contrib.sessions',

    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
    'accounts',
    'pyqs',
    'courses',]
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',]
ROOT_URLCONF = 'smartstudy.urls'
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages', ], },  },]
WSGI_APPLICATION = 'smartstudy.wsgi.application'
DATABASES = {
```

```
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3', }}
AUTH_PASSWORD_VALIDATORS = [
    {'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},]
REST_FRAMEWORK = {
    'DEFAULT_RENDERER_CLASSES': [
        'rest_framework.renderers.JSONRenderer',
    ],}
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
STATIC_URL = 'static/'
AUTH_USER_MODEL = 'accounts.User'
CORS_ALLOW_ALL_ORIGINS = True
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```